

Laboratorio de Estructura de Computadores

ELO 312 - Clase 02

Funciones, arreglos, punteros y estructuras en C

Alejandro Weinstein

(basado en material del profesor Ioannis Vourkas)

11 de agosto 2026, Valparaíso, Chile



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



DEPARTAMENTO DE
ELECTRONICA

Funciones en C

- Misma idea que en otros lenguajes de programación: “grupo de instrucciones que realizan una tarea, con una interfaz claramente descrita”.
- Cada función consta de una **declaración** y de una **definición**.
- La declaración indica el nombre de la función, el tipo de dato que retorna, y el tipo de dato de sus parámetros.

Declaración

Forma general:

```
return_type function_name (parameter list );
```

Ejemplo de una función que retorna el máximo entre dos enteros:

```
int max (int, int);
```

Opcionalmente:

```
int max (int a, int b); // forma preferente  
                        // es mas claro
```

Definición

Forma general:

```
return_type function_name( parameter list ) {  
    body of the function  
}
```

Ejemplo:

```
int max (int a, int b) {  
    int result;  
    if (a > b)  
        result = a;  
    else  
        result = b;  
    return result;  
}
```

Llamado de la función (ejemplo)

```
#include <stdio.h>

/* declaración de la función*/
int max (int, int);

int main (void) {
    /* definición de variables locales*/
    int a = 100;
    int b = 200;
    int ret;
    /* llamado de la funcion*/
    ret = max (a, b); // o max (b, a);
    printf ( "Max value is : %d\n", ret );
    return 0;
}
```

```
/* en el mismo archivo */

/* definición de la función */
int max (int a, int b) {
    int result;
    if (a > b)
        result = a;
    else
        result = b;
    return result;
}
```

Variables globales y locales

El **scope** de una variable es una región del programa donde una variable definida existe. Fuera de esa región no existe.

Las variables se pueden declarar:

- Dentro de una función o un bloque (variables locales)
- Fuera de todas las funciones (variables globales)

Variables locales

Solo pueden usarse por sentencias que están dentro de esa función o bloque de código. No son vistas/conocidas fuera del bloque o en otras funciones.

Variables globales

Las variables globales se definen fuera de una función, generalmente en la parte superior del programa. Mantienen sus valores a lo largo de la vida útil de su programa y se puede acceder a ellas dentro de cualquiera de las funciones definidas.

Convención: Su primera letra se escribe con mayúscula.

Arreglos

Los *arrays* (arreglos) son una estructura de datos para almacenar una colección secuencial y de tamaño fijo de elementos del mismo tipo.

Todos los arreglos consisten en ubicaciones de memoria contigua.

Declaración:

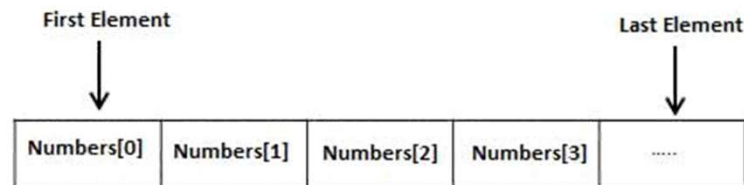
```
type array_name [array_size];
```

`array_size` debe ser una constante entera mayor que cero

`type` puede ser cualquier tipo de datos válido.

Arreglos

El primer elemento tiene índice 0



Ver <https://www.cs.utexas.edu/~EWD/transcriptions/EWD08xx/EWD831.html>

Arreglos

Ejemplo:

	0	1	2	3	4
balance	1000.0	2.0	3.4	7.0	50.0

```
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

```
int n[ 10 ];  
/* initialize elements of array n */  
for (int i = 0; i < 10; i++ ) {  
    n[i] = i + 100;  
}
```

Arreglos como parámetro de una función

Tres alternativas:

```
void mi_funcion (int *param) {  
    ...  
}
```

```
void mi_funcion (int param[10]) {  
    ...  
}
```

```
void mi_funcion (int param[]) {  
    ...  
}
```

Ejemplo:

```
#include <stdio.h>  
  
double get_average (int arr[], int size);  
  
int main () {  
    int balance[5] = {1000, 2, 3, 17, 50};  
    double avg;  
    avg = get_average (balance, 5) ; // puntero a balance  
    return 0;  
}  
  
double get_average (int arr[], int size) {  
    double sum = 0;  
    for (int i = 0; i < size; ++i) {  
        sum += arr[i];  
    }  
    return (sum / size);  
}
```

Arreglos

¿En qué se parece y en qué se diferencian los arreglos en C con las listas en Python?

Punteros

Motivación:

¿Cómo podemos definir una función que retorne un arreglo?

Punteros

Un puntero es una variable cuyo valor es la dirección de otra variable, es decir, la dirección directa de la ubicación en la memoria.

Declaración:

```
type *var_name;
```

Es equivalente a escribir `type* var_name;`

Ejemplo:

```
float *fp;
```

Punteros

Ejemplo:

```
#include <stdio.h>

int main (void) {
    int  var = 20;  /* variable declaration */
    int  *ip;       /* pointer declaration */

    /* store address of var in pointer*/
    ip = &var;

    printf("Address of var : %x\n", &var );
    printf("Address stored in ip : %x\n", ip );
    printf ("Value of *ip variable: %d\n", *ip );

    return 0;
}
```

El operador & entrega la dirección de una variable.

Hay dos maneras de usar un puntero:

- Sin * : Corresponde a la dirección de la variable a la que apunta.
- Con * : Corresponde al valor de la variable a la que apunta (“dereferencing”).

Aritmética de punteros

- Podemos usar ++, --, +, y -
- La operación aritmética opera en función del tamaño del tipo de datos al que apunta el puntero.
- Por ejemplo, si

`int *ptr; // supongamos que int son 4 bytes`

entonces

`ptr++;`

aumenta el valor de ptr en 4.

- ¿Cuál es la diferencia entre `ptr++` y `*ptr++`?

(Ayuda: revisar precedencia de los operadores

https://en.cppreference.com/w/c/language/operator_precedence.html)

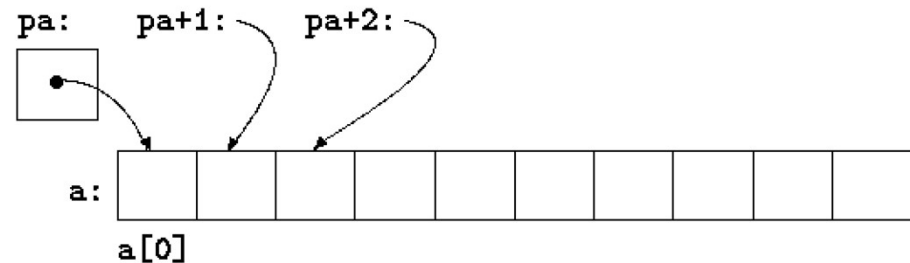
Punteros y arreglos

- El nombre de un arreglo es un puntero **constante** al primer elemento del arreglo.
- Por ejemplo, si definimos `double balance [50];` `balance` es un puntero a `&balance[0]`, la dirección del primer elemento del arreglo.

- Ejemplo

```
double* pa;  
double a [10];  
pa = a;
```

- Podemos usar nombres de arreglos como punteros constantes, y viceversa. Por lo tanto, `*(a + 4)` es una forma de acceder a `a[4]`.
- Es decir `*pa`, `*(pa + 1)`, `*(pa + 2)`, ... son equivalentes a `a[0]`, `a[1]`, `a[2]`, ...



Punteros y arreglos

```
#include <stdio.h>
```

```
int main ( ) {  
    double arr[5] = {10.0, 2.0, 3.4, 17.0, 50.0};  
    double *p;  
    p = arr;  
    for (int i = 0; i < 5; i++ ) {  
        printf("(p + %d) : %f\n", i, *(p + i) );  
    }  
    printf("Array values using arr\n");  
    for (int i = 0; i < 5; i++ ) {  
        printf("(arr + %d) : %f\n", i, *(arr + i) );  
    }  
  
    return 0;  
}
```

Funciones: Parámetros por valor y por referencia

Por valor

```
/* function definition to swap the values */  
void swap (int x, int y) {  
  
    int temp;  
  
    temp = x; /* save the value of x */  
    x = y;    /* put y into x */  
    y = temp; /* put temp into y */  
  
    return;  
}
```

Por referencia

```
/* function definition to swap the values */  
void swap (int *x, int *y) {  
  
    int temp;  
  
    temp = *x;    /* save the value at address x */  
    *x = *y;      /* put y into x */  
    *y = temp;    /* put temp into y */  
  
    return;  
}
```

Sólo una de estas dos versiones funciona ¿Cuál?

Funciones: Parámetros por valor y por referencia

```
#include <stdio.h>

void swap (int *x, int *y);

int main ( ) {
    int a=5, b=10;
    printf("a = %d ; b = %d \n", a, b);
    swap(&a, &b); // Notar que usamos &
    printf("a = %d ; b = %d \n", a, b);
    return 0;}
```

Estructuras

- Son una manera de organizar datos relacionados entre sí.

```
struct structure_tag {  
    member definition;  
    member definition;  
    ...  
    member definition;  
} one or more structure variables;
```

- Ejemplo:

```
struct Book {  
    char  title [50];  
    char  author [50];  
    char  subject [100];  
    int   id;  
} book_1;  
book_1.id = 42;
```

Estructuras

```
#include <stdio.h>
#include <string.h>

struct Book {
    char *title;
    char author [50];
};

void printBook (struct Book);

int main( ) {
    struct Book B1; /* structure of type Book */
    B1.title = "C Programming";
    strcpy( B1.author, "Nuha Ali");

    printBook (&B1);
    return 0;
}

void printBook ( struct Book *b ) {
    printf ( "Book title : %s\n", b->title);
    printf ( "Book author : %s\n", b->author);
}
```

Igual que para cada otro tipo de datos, se pueden definir **punteros a estructuras**:

```
struct Book *struct_pointer;
```

La dirección de una estructura la obtenemos con el &:

```
struct_pointer = &B1;
```

Ahora podemos usar el puntero para acceder a los datos miembros de la estructura

```
(*struct_pointer).title = "ELO 312";
```

Con punteros a estructuras, es más conveniente usar el
-> en lugar de (*)

```
struct_pointer->title = "STM32";
```